

Cómo crear loops en Claude Code con comandos



Cinco comandos para trabajar de forma continua: memoria del proyecto, ciclos recurrentes y gestión del contexto.

1 Los cinco comandos clave

`/init` › `/memory` › `/loop` › `/compact` › `/clear`

`/init` · Inicializa el proyecto

Genera `CLAUDE.md` con arquitectura, estructura, comandos, frameworks y reglas generales.

`/memory` · Configura la memoria

Edita `CLAUDE.md` para fijar lo que Claude debe respetar en todas las sesiones.

`/loop` · Crea el ciclo

Repite un prompt a intervalos dentro de la sesión, apoyándose en la memoria del proyecto.

`/compact` · Compacta el contexto

Resume la conversación conservando lo relevante para seguir sin perder el hilo.

`/clear` · Limpia la sesión

Borra el historial de la conversación; `CLAUDE.md` se mantiene intacto.

2 Qué incluir en la memoria (CLAUDE.md)

Frameworks y librerías

Convenciones de código

Comandos de build y test

Reglas de arquitectura

Restricciones y buenas prácticas

3 Sintaxis y ejemplos de `/loop`

Formato: `/loop [intervalo] <prompt>` . Intervalos del tipo `30s` , `5m` , `1h` .

```
/loop                                ciclo con el intervalo por defecto
/loop 5m revisa el estado del despliegue    cada 5 minutos
/loop corrige los errores de TypeScript y ejecuta los tests hasta que pasen todos
/loop revisa el proyecto, corrige vulnerabilidades y verifica que no queden incidencias
```

4 Cómo funciona cada iteración

Analizar estado › Elegir tarea › Implementar solo esa › Revisar código › Detectar fallos › Corregir › Resumir › Repetir

El ciclo continúa hasta cumplir el criterio de parada definido en el prompt.

5 ¿Compactar o limpiar?

`/compact` — cuándo usarlo

Conversación muy larga · para reducir el tamaño del contexto · antes de iniciar un nuevo loop.

`/clear` — cuándo usarlo

Al cambiar de objetivo por completo · al empezar una tarea nueva · al cambiar de proyecto.

6 Criterio de parada y buenas prácticas

Define cuándo termina el loop

Funcionalidades implementadas · sin errores de compilación · todas las pruebas en verde · cobertura adecuada · documentación completa · requisitos cumplidos.

Buenas prácticas

Objetivo concreto · memoria actualizada · validar antes de pasar a la siguiente tarea · no salirse del alcance sin justificarlo · revisar los resúmenes de cada iteración.

Matrices a tener en cuenta

- `/loop` es **de sesión**: se detiene al interrumpirlo (`Ctrl+C`) o al cerrar Claude Code. Para tareas persistentes conviene usar tareas programadas o CI.
- El loop no "sabe" cuándo parar por sí solo: **escribe el criterio de parada en el propio prompt**.
- Cada iteración consume contexto; combinarlo con `/compact` ayuda en ciclos largos.

En resumen · `/init` y `/memory` fijan el contexto estable, `/loop` automatiza el trabajo repetitivo y `/compact` / `/clear` mantienen la sesión ligera: usados con estrategia, convierten Claude Code en un **asistente de desarrollo continuo y consistente**.